Introduction to Data Analytics and Modeling

John R. Crooker, PhD

Thursday, October 22, 2026

Table of contents

Objectives . . . 1
Our AI-assisted analytics workflow . . . 1
Some uses of Data Analytics and models . . . 2
Data . . . 3
Claude Analytics Laboratory 1: Obtaining data from a data service . . . 5
Visualizing the data . . . 9
Selecting versus calculating: an audit . . . 10
Cross-sectional data . . . 12
Branches in statistics . . . 18
Ethics and statistics — it's a dangerous world of data out there . . . 22

Objectives

- How statistics is used in the business world.
- About the sources of data and the methods for collecting it.
- How to classify data by the measurement approach.
- To distinguish between time series and cross-sectional data.
- To distinguish between descriptive, inferential and predictive statistics.
- About the ethical implications of misusing statistics.
- Organize a reproducible, AI-assisted analysis.
- Use **Claude** and **Python** to retrieve economic data from the Federal Reserve Economic Data (**FRED**) service.
- Verify the identity, units, frequency, date range, and missing values of downloaded data.
- Record the provenance of every data set you analyze.

Our AI-assisted analytics workflow

This semester we will use **Claude** as an analytical collaborator. Claude can help us write, run, explain, and debug computer code, and it can execute that code and show us the result. It can often complete calculations much faster than we can complete them manually. However, speed does not guarantee that the data, method, or interpretation is appropriate.

Our work will follow this sequence:

1. **Define the business question.** What decision or claim will the analysis support?
2. **Specify the data.** What does each observation represent? Which variables, dates, units, and populations are relevant?
3. **Direct Claude.** Give Claude a precise task, relevant constraints, and a required output.
4. **Run reproducible code.** Preserve the code and record where the data came from.
5. **Verify the work.** Inspect the data, calculations, assumptions, and output instead of accepting them automatically.

6. **Interpret and communicate.** Translate results into a defensible business conclusion and disclose important limitations.

> **Claude is a capable junior analyst, not the decision maker.** You are responsible for the analytical specification, verification, interpretation, and final recommendation.

The one rule that governs everything else

> **Every number you report must come from code that was actually executed.** Never take a figure from an assistant's prose. If you cannot point to the cell that produced a number, the number does not go in your submission.

An assistant that writes a paragraph containing plausible-looking values has not performed an analysis. An assistant that runs a script and shows you its output has. The difference is invisible in the writing and decisive in the result.

Organizing an analysis

You will do most of your work this semester in Claude's code-execution environment: you upload a data file and a starting script, and Claude runs Python on them and shows you the output. There is nothing to install.

Even in that setting, keep the same discipline a full project folder would impose:

```
Inputs      -- the data file exactly as it was downloaded, unmodified
Program     -- the script that transforms inputs into results
Outputs     -- tables and figures, always regenerated by running the program
Provenance  -- a written record of where the data came from
```

We do not silently overwrite or hand-edit source data. If a number needs to change, the program changes, and we run it again. This is what makes an analysis *reproducible*: someone else can start from your inputs and your program and arrive at your outputs.

A provenance record is short and it is not optional:

```
Series:      DFF - Federal Funds Effective Rate
Source:      Federal Reserve Economic Data (FRED), Federal Reserve Bank of St. Louis
URL:         https://fred.stlouisfed.org/series/DFF
Retrieved:   2026-08-24
Frequency:   Daily, 7-day
Units:       Percent, not seasonally adjusted
Date range:  2020-01-01 through 2026-08-21
Rows:        2,425
```

Some uses of Data Analytics and models

Marketing research

Taste tests have been a staple of marketing research. Kellogg researchers, for example, would provide consumers with **Cheez-It** crackers and **Nabisco's Cheese Nips**. The researchers would ask consumers to rate the product. By repeating this research with thousands of consumers, the researchers would establish trends in preferences across the market. This research would shape product development. If you avoid the researchers armed with clipboards in the mall (and everyone like you with similar tastes), your preferences will not be reflected in the next evolution of snack food!

It is actually much more difficult to elude the market researchers today. Many websites you visit are monitoring **clickthrough rates** (CTR). We will say much more about **marketing research** in this class.

Advertising

The television industry for decades has used **Nielsen Media Research**. These ratings are based on data collected from a panel of US households. The Nielsen researchers sample from these households to measure viewing habits and preferences for the entire country. The design of these samples is critical in ensuring that the results from the survey responses reflect the nation as a whole. **Sampling strategies** and **statistical inferences** are a topic for us this semester.

Operations

Quality control is an important consideration for today's large scale producers. As many production processes are **automated**, sampling from the large quantities of goods produced in each period is a technique to assess production quality. As no production process is perfect, quality control engineers establish benchmarks for inherent variation in, say, product dimensions. **Six Sigma** is a methodology intent on reducing this **inherent variation** to a small or minuscule level. Our quality control engineers identify problems in the production processes when they find production deformities outside the defined benchmarks. Statistical **variation** is an important topic for our investigations.

Finance

Many economic events cannot be perfectly anticipated. This is particularly troublesome given the experience of some economic shocks can have disruptive change on a business. Moreover, many of these events cannot be avoided. For example, if the global economy enters a recession, **pro cyclical** businesses would more than likely experience sharp reductions in consumer demand. Decisions regarding launching a new product line or new manufacturing facility could have disastrous implications for a business if these launches coincided with a recession. Finance professionals use **probability** as a tool to weight potential outcomes across all potential business climate scenarios. As economic conditions change, these probabilities are reassessed and the **average** or **expected returns** from a business decision are recalculated. Modern statistical spreadsheets and statistical packages are used to **cascade** the changing probabilities across all upcoming or reversible decisions of the business.

Sports business

Insights regarding player performance that impacts team outcomes can be exploited by a team if they can recognize undervalued traits in players. Famously, the Oakland Athletics front office did exactly this by recognizing that a statistical analysis identified players who were having large impacts on positive team performance but were not being compensated for these contributions (see Moneyball). This provided an opportunity for the organization to buy and **play** this undervalued talent.

Data

> **Data** are values assigned to observations or measurements and are the building blocks of statistical analysis.

> **Information** is data that are transformed into useful facts that can be used for a specific purpose, such as making a decision.

> A **data set** is a collection of data points in a single application.

Structured data are data that has been ordered into rows and columns (a data table). The rows represent a distinct element (i.e., a client or a person). The columns represent different characteristics that have been measured for the distinct element (i.e., street address, postal code, vertical jump).

Unstructured data are data that is not ordered into data tables. Examples include the characters in a tweet, the text from a news break, and the transcript of a corporate report.

Secondary data are data that somebody else has collected and made available for others to use.

Primary data are data that you have collected for your own use.

Direct observation is a method of gathering data while the subjects of interest are in their natural environment, often unaware that their behavior is being recorded.

A **focus group** is a direct observational technique whereby individuals are often paid to discuss their attitudes toward products or services in a group setting controlled by a moderator.

An **experiment** is a method of gathering primary data by exposing subjects to certain treatments and recording the data of interest.

A **survey** is a method of collecting primary data by directly asking people a series of questions. They can be administered by e-mail, via the web, through snail mail, face to face, or over the telephone.

Two main types of data

Quantitative data use numerical values to measure a characteristic of interest.

Qualitative data use non-numerical values to assign labels to a characteristic. For example, level-of-education may be the degree earned: Less than High School, High School Diploma, Associates Degree, Bachelors Degree, Masters Degree, Doctorate.

Classifying data by their measurement approach

Nominal data are data that are described as a category or a label. Examples are marital status (married, single, divorced, widowed), or membership status (yes, no).

Ordinal data have all the properties of nominal data, with the added feature that we can rank-order the values from highest to lowest.

Interval data is strictly quantitative and allows us to measure the differences between the categories with actual numbers in a meaningful way. For example, temperatures, or product dimensions.

> **Ratio data** have all the features of interval data with the added benefit of having a true zero point. *True zero*: If the phrase "twice as much" accurately describes the relationship between two values that differ by a multiple of two, the data can be considered *ratio data*.

Type	Key Properties	Examples	Notes
Nominal	• Categories / labels only - No order or ranking	• Marital status: Married, Single, Divorced, Widowed - Membership: Yes / No - Hair color: Brown, Black, Blonde	Numbers (if used) are just codes, not meaningful values.
Ordinal	• Categories **with order** - Differences between ranks not meaningful	• Education level: High School < College < Graduate < PhD - Likert scale: Strongly Disagree → Strongly Agree - Race positions: 1st, 2nd, 3rd	Can say "higher/lower," but can't quantify "how much higher."
Interval	• Numeric scale - Differences are meaningful - **No true zero** (ratios don't make sense)	• Temperature (°C, °F) - Calendar years - IQ scores - Time of day (clock)	Can add/subtract, but "twice as much" is not valid.
Ratio	• All interval properties - **True zero** (absence of what's measured) - Ratios meaningful	• Age - Weight - Height - Income - Temperature in Kelvin	Can multiply/divide: "twice as much" makes sense.

Time series versus cross-sectional data

> **Time series data** are values that correspond to specific measurements taken over a range of time periods.

The **Federal Funds Effective Rate** is the interest rate at which depository institutions trade federal funds (balances held at Federal Reserve Banks) with each other overnight. The FRED series identifier is **DFF**. Source: Federal Funds Effective Rate on FRED.

Claude Analytics Laboratory 1: Obtaining data from a data service

An **application programming interface (API)** is a documented way for one computer program to request data or services from another. Instead of downloading and editing a spreadsheet manually, we can write a reproducible program that requests a defined FRED series. The program can be re-run tomorrow and will pick up tomorrow's data; a spreadsheet you edited by hand cannot.

FRED exposes its data two ways, and the distinction matters for this course:

- A **public CSV endpoint** that requires no credentials. It is an ordinary URL that returns ordinary CSV:

  ```
  https://fred.stlouisfed.org/graph/fredgraph.csv?id=DFF&cosd=2020-01-01
  ```

  Paste that into a browser and you will get the data. This is what we will use for lecture work and Laboratory 1, because every student can run it immediately with nothing installed but `pandas`.

- A **full REST API** at `api.stlouisfed.org` that requires a free personal API key and returns richer metadata. You will use this later in the Group Forecast Project. See the FRED API documentation.

The retrieval itself is done once, by your instructor, using a small course module called `fred_tools.py` — it lives in `tools/` in the course repository if you want to read it. It converts FRED's `.` missing-value marker to `NaN`, enforces the date and numeric types, and raises a clear error when a series ID is wrong rather than returning a confusing half-parsed table.

You will not run it. Your exercises read the CSV it produced, which is committed to the repository along with a record of exactly where each series came from. That is deliberate: an analysis that depends on a live network connection fails in ways that have nothing to do with the analysis.

> When you do move to the keyed API, an API key is a **credential**. Never paste a key into a prompt, a notebook, a rendered report, or anything you submit. If a key is ever exposed, revoke it and request a new one.

Our first task for Claude

Everything this course needs lives in a public repository. There are two ways to get it in front of Claude, and it is worth knowing both before you start.

Path 2 below is the elegant one — you tell Claude where the files are and it fetches them. It only works if your account's web access is on, and even then Claude cannot always retrieve a file by direct link. **Path 1 works everywhere.** Neither is more correct than the other; they differ only in who does the fetching.

Path 1 — upload the files

Download the Laboratory 1 pack — one file. Unzip it and you will have a folder holding seven files: the three scripts used in this chapter, the helper they share, two data files, and the course rules. Drag all seven into a new Claude conversation, wait for each to show as attached, and paste this:

> **PROMPT**
>
> I have uploaded seven files for a graduate business statistics course. Read `COURSE_CONTEXT.md` first — it explains how this course expects analytical work to be done.
>
> Then run `01a_inspect_dff.py` and show me its actual output, verbatim. Walk me through what each section does and what the results mean. Do not report any number you did not compute, and if anything fails, show me the error rather than describing what the output would have been.

Path 2 — let Claude fetch the files

Open a Claude conversation and paste this:

> **PROMPT**
>
> Open https://crooker.faculty.unlv.edu/mba775/fall2026/upload_packs/ — it is a plain page listing the files for each laboratory. Follow the links for Laboratory 1 and read `COURSE_CONTEXT.md` first; it explains how this course expects analytical work to be done.
>
> Then run `01a_inspect_dff.py` and show me its actual output, verbatim. Walk me through what each section does and what the results mean. Do not report any number you did not compute, and if anything fails, show me the error rather than describing what the output would have been.
>
> If you cannot open that page, tell me plainly and I will upload the files instead.

Notice what this prompt specifies:

- **where the materials are**, so nothing depends on you managing files;
- **exactly which files** are needed, rather than "the course stuff";

- **the standing rules**, by pointing at `COURSE_CONTEXT.md` instead of re-explaining them every session;
- that the code must **actually be executed**, not described;
- that reported numbers must come from that execution; and
- **what to do when something fails** — which is the line that keeps a stuck session from turning into a wrong answer delivered confidently.

That last point is worth dwelling on. An assistant asked to produce output will usually produce something. Telling it what to do when it *cannot* is how you find out that it couldn't.

If Claude says it cannot reach GitHub

You will see a reply along these lines:

> I don't have direct network access in this environment, and searching for the repository didn't surface it. If you upload the files, I'll run the script and show you the actual output.

Nothing is wrong. Not with your work, not with the repository, and not with Claude. That answer is the prompt working exactly as written — the last line asked Claude to say so plainly instead of inventing results, and it did.

Two things cause it, and they are worth telling apart.

Web access may simply be switched off. It is a setting, not a default. To check: open **claude.ai**, click your name or initials in the lower-left corner, choose **Settings**, and look for the web search or web access toggle. If it is off, turn it on and start a *new* conversation — a conversation that began without web access does not gain it partway through.

Even with it on, a direct file link may not be fetchable. Claude's web tools follow links found on pages they have already loaded, and generally refuse to open a URL constructed from scratch. That is why Path 2 points at a plain page of links on the course site rather than at a code repository — repositories serve their file listings through pages that block automated fetching, and a filename printed as text is not a link anything can follow.

Either way, the fix is the same: use Path 1. It needs no settings, no search, and no permission — and it is how most of this class will run the labs.

The setting that actually matters

There is a second setting, and it matters far more than the first one.

Under **Settings → Capabilities**, find **Code execution and file creation** and make sure it is **on**. On the Free, Pro, and Max plans you have to turn this on yourself; it is not enabled by default.

> Without it, Claude cannot run anything. It will read the script, understand it perfectly well, and *describe* what the output would look like — in fluent, confident prose, with numbers that came from nowhere.
>
> That is not a broken lab. That is the exact failure this entire course is organized against, arriving disguised as success.

How to tell the difference. When code execution is on, you will see the code actually run — a panel or block showing the script executing, and printed output appearing beneath it. When it is off, you get prose. If Claude tells you the series has 26,342 rows but you cannot find a line of output that *says* 26,342, the number is not evidence. Ask directly:

> Did you actually execute that script, or are you describing what it would produce? Show me the raw output.

An honest assistant will tell you. Then turn the setting on and start a new conversation.

> Do not let a stuck session turn into a fabricated answer. If Claude cannot get the files, it should tell you — and if it ever reports numbers *without* showing you the executed output that produced them, that is the failure this whole course is organized against.

Where the data came from

You are analyzing a CSV, but somebody had to retrieve it. This is the code that produced `data/dff.csv`, run once by your instructor rather than by each of you thirty times:

```
from fred_tools import fred_series

dff = fred_series("DFF", start="2020-01-01").to_frame("federal_funds_rate")

missing_count = int(dff["federal_funds_rate"].isna().sum())

print(f"Downloaded {len(dff):,} DFF observations.")
print(f"Date range: {dff.index.min().date()} to {dff.index.max().date()}")
print(f"Missing rate values: {missing_count:,}")

dff.to_csv("dff_downloaded.csv")
print("Saved data to dff_downloaded.csv")
```

```
Downloaded 2,417 DFF observations.
Date range: 2020-01-01 to 2026-08-13
Missing rate values: 0
Saved data to dff_downloaded.csv
```

```
dff.head()
```

date	federal_funds_rate
2020-01-01	1.55
2020-01-02	1.55
2020-01-03	1.55
2020-01-04	1.55
2020-01-05	1.55

Verification checklist

Do not stop when the program runs without an error. Verify that:

- the file really is `DFF` — the Federal Funds Effective Rate — and not some neighbouring series with a similar name;
- the index is stored as dates and the rate column is numeric;
- the dates are **unique** and in order, before you trust any count or average computed from them;
- FRED's `.` missing-value marker became a missing numeric value rather than the text `"."`;
- the script reports rather than silently deletes missing observations;
- the row count and latest date are plausible against the FRED webpage for DFF; and
- every number in your write-up appears somewhere in the executed output above.

The third item is the one students skip. A duplicated date inflates every count and can offset a genuine gap, so a coverage check that looks reassuring may be measuring a broken series. Verify the thing you are measuring *with*, not just the thing you are measuring.

> A data request that succeeds establishes only that the server returned something. It does not establish that we requested the correct concept, units, frequency, dates, or transformation. Those are analytical responsibilities.

Here is the check on data types, which is the one students skip most often:

```
print(dff.dtypes)
print(f"\nIndex type: {type(dff.index).__name__}")
print(f"Any non-numeric values coerced to NaN: {missing_count}")
```

```
federal_funds_rate    float64
dtype: object

Index type: DatetimeIndex
Any non-numeric values coerced to NaN: 0
```

Visualizing the data

Every figure in this note is drawn with `_charts.py` — the same module that comes in your laboratory pack. That is deliberate: code you copy out of this note runs in your environment, because it uses nothing you do not have.

> The chunk below calls `fred_series()`, which lives in `tools/fred_tools.py` — **instructor tooling, not part of your laboratory pack.** It is shown so you can see where the data came from, which is this chapter's subject. Do not copy it into your own session; it will fail with a `NameError`, because that module is not among the files you were given. Your scripts read the local `dff.csv` instead, which is the reproducibility discipline this chapter argues for.

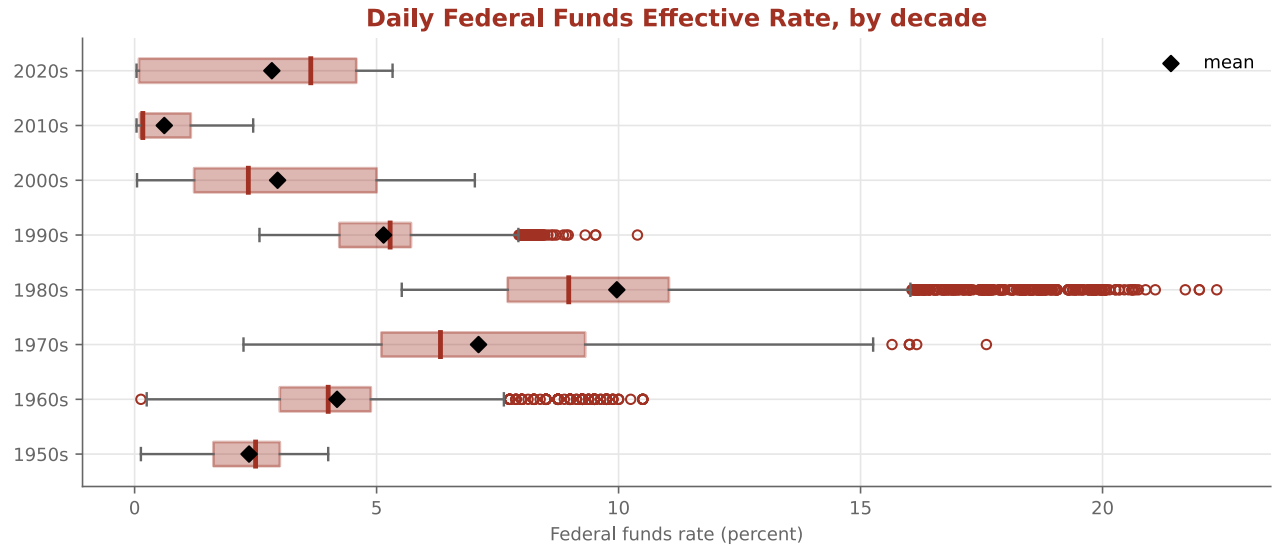
```
dff_full = fred_series("DFF").to_frame("DFF")
dff_full.index.name = "Date"
print(f"Full history: {len(dff_full):,} observations, "
      f"{dff_full.index.min().date()} to {dff_full.index.max().date()}")
```

```
Full history: 26,342 observations, 1954-07-01 to 2026-08-13
```

Daily Federal Funds Rate by Decade

```
my_dff = dff_full.reset_index()
my_dff["Decade"] = my_dff["Date"].dt.year.floordiv(10).mul(10).astype(str) + "s"

decades = sorted(my_dff["Decade"].unique())
_ = box_plot([my_dff.loc[my_dff["Decade"] == d, "DFF"] for d in decades],
             labels=decades,
             title="Daily Federal Funds Effective Rate, by decade",
             xlab="Federal funds rate (percent)")
```



Selecting versus calculating: an audit

The previous version of this note built a sequence of month-start dates and used it to index the daily series directly. In R that looked like `dff[my_months, ]`; the Python equivalent is a `.loc` lookup on a list of dates. Both do the same thing, and it is worth being precise about what that thing is.

```
my_months = pd.date_range("2020-01-01", pd.Timestamp.today().normalize(), freq="MS")
selected = dff.reindex(my_months)
selected.head(12)
```

	federal_funds_rate
2020-01-01	1.55
2020-02-01	1.59
2020-03-01	1.58
2020-04-01	0.06
2020-05-01	0.05
2020-06-01	0.05
2020-07-01	0.08
2020-08-01	0.10
2020-09-01	0.09
2020-10-01	0.09
2020-11-01	0.09
2020-12-01	0.09

> This **selects particular daily observations** whose dates happen to be the first of a month. It does **not calculate** a monthly federal funds rate.

There is a second, quieter hazard in selecting by date. A requested date only returns a value if the series actually has an observation on that day, and whether it does depends on the series' publication calendar. Rather than assume, we check:

```
n_requested = len(my_months)
n_missing = int(selected["federal_funds_rate"].isna().sum())

print(f"Month-start dates requested: {n_requested}")
print(f"Dates with no observation:   {n_missing}")
if n_missing:
    print("Missing:", [d.date() for d in
                       selected.index[selected['federal_funds_rate'].isna()]][:12])
```

```
Month-start dates requested: 82
Dates with no observation:   2
Missing: [datetime.date(2026, 9, 1), datetime.date(2026, 10, 1)]
```

`DFF` is published on a 7-day daily calendar, so it has a value on weekends and holidays and this lookup happens to find one every month. Run the same code against a business-day series — `DGS10`, say, or `SP500` — and month-starts that land on a weekend come back empty. Selecting by date is fragile in a way that is invisible until the calendar changes underneath you.

Three defensible alternatives

```
rate = dff["federal_funds_rate"]

monthly = pd.DataFrame({
    "mean":  rate.resample("MS").mean(),
    "first": rate.resample("MS").first(),
    "last":  rate.resample("MS").last(),
})
monthly.head(12).round(4)
```

date	mean	first	last
2020-01-01	1.5506	1.55	1.59
2020-02-01	1.5831	1.59	1.58
2020-03-01	0.6523	1.58	0.08
2020-04-01	0.0490	0.06	0.05
2020-05-01	0.0500	0.05	0.05
2020-06-01	0.0777	0.05	0.08
2020-07-01	0.0926	0.08	0.10
2020-08-01	0.0952	0.10	0.09
2020-09-01	0.0900	0.09	0.09
2020-10-01	0.0900	0.09	0.09
2020-11-01	0.0863	0.09	0.09
2020-12-01	0.0900	0.09	0.09

Each answers a different business question:

- the **monthly mean** describes average borrowing conditions *during* the month;
- the **first available observation** describes the rate the month *opened* with; and
- the **last available observation** describes the rate the month *closed* with.

These calculations produce different numbers. The correct transformation depends on the question, and choosing one without stating the question is not a technical shortcut — it is an unstated assumption.

```
spread = (monthly["mean"] - monthly["last"]).abs()
print(f"Largest gap between the monthly mean and the month-end rate: "
      f"{spread.max():.3f} percentage points, in "
      f"{spread.idxmax().strftime('%B %Y')}.")
```

```
Largest gap between the monthly mean and the month-end rate: 0.644 percentage points, in July 2022.
```

Cross-sectional data

> **Cross-sectional data** are values collected from a number of subjects (firms, individuals, states, regions, etc.) during the same time period (or nearly the same).

State unemployment levels can also be retrieved from **FRED**. Conveniently, the *symbol* for the unemployment rate for a state is the state's two letter abbreviation followed by the letters "UR". The Nevada unemployment rate is **NVUR**.

Extending the workflow to cross-sectional data

The definition of a cross section requires more care than merely downloading 50 series. State unemployment observations are released on schedules that can differ, and the latest non-missing date need not be identical for every state. A defensible comparison must establish a common reference date or explicitly document any timing differences.

The Laboratory 1 pack already contains this script and its data — you do not need to download anything else. In the same conversation where you ran `01a_inspect_dff.py`, ask Claude:

> **PROMPT**
>
> Now run `01c_state_cross_section.py` from the files I uploaded. Show me the actual output. Then answer two questions separately, and do not merge them:
>
> First: does every state's most recent observation fall on the same date? If the dates differ, report the differences and ask me to choose a common-date rule rather than silently combining unlike periods.
>
> Second: are there any missing values anywhere in this series — not just at the end? If so, tell me how many, which states, and which dates. Do not drop them silently.

Run it, and then read the two answers against each other.

The first check passes. Every state reports through the same month, so this really is a clean cross section and there is no rule for you to choose.

The second check does not pass. Look at what it found.

That gap is the lesson. The date-alignment check examines each state's **latest** observation, and the hole is not at the end of the series — it is in the middle. No month is absent from the file, so a row count cannot see it either: the row exists, and the value inside it is empty.

> **A check that passes is not the same as data that is clean.** It means the question you asked came back fine. A different question, asked of the same file, comes back differently.
>
> This is why `01a_inspect_dff.py` reports a missing-value count rather than assuming zero, and why the script you just ran prints how many observations it is about to discard instead of discarding them quietly. An analysis that drops data without saying so is not wrong in its arithmetic. It is wrong about what it is describing — and nobody reading the output can tell.
>
> `data/README.md` documents this gap and says plainly that it is how FRED distributes the series. Missing months in official statistics usually have a reason, and finding out what it was is a better exercise than filling the hole.

> This task introduces the habit the rest of the course depends on: **Claude should surface a consequential analytical choice rather than quietly make it for us.**
>
> Notice that the prompt does not merely ask for the answer. It asks Claude to *stop* at a specific point and hand a decision back. An assistant left to its own devices will pick a rule — probably a reasonable one — and never mention that a choice was made. The unstated assumption is the danger, not the rule itself.

```
state_abb = [
    "AL", "AK", "AZ", "AR", "CA", "CO", "CT", "DE", "FL", "GA",
    "HI", "ID", "IL", "IN", "IA", "KS", "KY", "LA", "ME", "MD",
    "MA", "MI", "MN", "MS", "MO", "MT", "NE", "NV", "NH", "NJ",
    "NM", "NY", "NC", "ND", "OH", "OK", "OR", "PA", "RI", "SC",
    "SD", "TN", "TX", "UT", "VT", "VA", "WA", "WV", "WI", "WY",
]
print(f"{len(state_abb)} states")
```

```
50 states
```

R supplies `state.abb` as a built-in constant. Python does not, so we write the list out. That is a small thing, but it is representative: several conveniences that R bundles for statistical work have to be supplied explicitly in Python.

```
rows = []
failures = []

for abb in state_abb:
    sid = f"{abb}UR"
    try:
        s = fred_series(sid, start="1976-01-01").dropna()
        rows.append({
            "State": abb,
            "Series": sid,
            "Date": s.index[-1],
            "UR": float(s.iloc[-1]),
        })
    except Exception as exc:
        failures.append({"State": abb, "Series": sid, "error": type(exc).__name__})

my_datfr = pd.DataFrame(rows)
```

```
print(f"Retrieved {len(my_datfr)} of {len(state_abb)} state series.")
if failures:
    print("Failures:", failures)
```

```
Retrieved 50 of 50 state series.
```

Are these observations actually contemporaneous?

Before we treat this as a cross section, we check the assumption the term depends on:

```
date_counts = my_datfr["Date"].value_counts().sort_index()
print(date_counts.to_string())

if len(date_counts) == 1:
    print(f"\nAll {len(my_datfr)} states report the same date. "
          f"This is a clean cross section.")
else:
    print(f"\nWARNING: {len(date_counts)} distinct observation dates. "
          f"These states are NOT all measured in the same period.")
    common = date_counts.index.max()
    print(f"States not reporting on {common.date()}: "
          f"{sorted(my_datfr.loc[my_datfr['Date'] != common, 'State'])}")
```

```
Date
2026-06-01    50

All 50 states report the same date. This is a clean cross section.
```

If that check reports more than one date, we do not proceed by ignoring it. We either restrict to a common date or we document the discrepancy in the write-up.

```
my_datfr.sort_values("UR", ascending=False).reset_index(drop=True)
```

	State	Series	Date	UR
0	CT	CTUR	2026-06-01	5.2
1	CA	CAUR	2026-06-01	5.2
2	OR	ORUR	2026-06-01	5.2
3	WA	WAUR	2026-06-01	5.2
4	IL	ILUR	2026-06-01	5.1
5	NV	NVUR	2026-06-01	5.1
6	MI	MIUR	2026-06-01	5.0
7	DE	DEUR	2026-06-01	4.9
8	AZ	AZUR	2026-06-01	4.9
9	NM	NMUR	2026-06-01	4.8
10	KY	KYUR	2026-06-01	4.7
11	FL	FLUR	2026-06-01	4.7

	State	Series	Date	UR
12	NY	NYUR	2026-06-01	4.6
13	NJ	NJUR	2026-06-01	4.5
14	MA	MAUR	2026-06-01	4.4
15	AK	AKUR	2026-06-01	4.4
16	LA	LAUR	2026-06-01	4.4
17	SC	SCUR	2026-06-01	4.4
18	MN	MNUR	2026-06-01	4.4
19	TX	TXUR	2026-06-01	4.4
20	MD	MDUR	2026-06-01	4.3
21	WV	WVUR	2026-06-01	4.2
22	OK	OKUR	2026-06-01	4.2
23	AR	ARUR	2026-06-01	4.1
24	PA	PAUR	2026-06-01	4.1
25	RI	RIUR	2026-06-01	4.1
26	CO	COUR	2026-06-01	3.9
27	MS	MSUR	2026-06-01	3.8
28	KS	KSUR	2026-06-01	3.8
29	VA	VAUR	2026-06-01	3.7
30	MO	MOUR	2026-06-01	3.7
31	ID	IDUR	2026-06-01	3.7
32	OH	OHUR	2026-06-01	3.6
33	UT	UTUR	2026-06-01	3.6
34	NC	NCUR	2026-06-01	3.6
35	TN	TNUR	2026-06-01	3.5
36	GA	GAUR	2026-06-01	3.4
37	IN	INUR	2026-06-01	3.3
38	WI	WIUR	2026-06-01	3.3
39	MT	MTUR	2026-06-01	3.3
40	AL	ALUR	2026-06-01	3.2
41	IA	IAUR	2026-06-01	3.2
42	WY	WYUR	2026-06-01	3.2
43	ME	MEUR	2026-06-01	3.1
44	NH	NHUR	2026-06-01	2.9
45	NE	NEUR	2026-06-01	2.9
46	HI	HIUR	2026-06-01	2.6

	State	Series	Date	UR
47	VT	VTUR	2026-06-01	2.6
48	ND	NDUR	2026-06-01	2.3
49	SD	SDUR	2026-06-01	2.0

This is an example of a *cross-sectional* data set. We have the prevailing state unemployment rate for Monday June 01, 2026 for all 50 US States.

Laboratory 1 deliverable

Submit four things. Do not submit the scripts or the data files — I wrote those and I already have them. What I want is what *you* established.

1. A data audit.

`data/README.md` in the course materials makes specific, checkable claims about `dff.csv`. Check each one against output you actually ran, and report the result as a table:

Claim in `data/README.md`	What your executed output showed	Agree?
Series is `DFF`, Federal Funds Effective Rate		
Units: percent, not seasonally adjusted		
Frequency: daily, 7-day — a value on every calendar day		
Date range		
Row count		
Missing values		

For each row, the middle column must be traceable to something Claude printed. If a claim and the output disagree, say which, by how much, and what you would do about it before using the data.

> Two of those claims are harder than they look. "A value on every calendar day" is not established by a row count — you have to check that no calendar date between the first and last observation is absent. And "not seasonally adjusted" cannot be verified from the file at all; it is a property of how FRED published the series, and the only way to check it is to go look. Say so if you cannot verify something from the data. **"I could not confirm this from the file" is a complete and correct answer.**

2. Your prompts.

The prompt you started with, plus anything else you had to ask for and why. The interesting part is the second half: where did the canned prompt prove insufficient, and what did you have to say to get what you needed?

3. The check that passed, and the one that did not.

When you ran `01c_state_cross_section.py`, the date-alignment check came back clean — every state reports through the same month. Report that result, and then report what the missing-value check found in the same file.

Then answer this in two or three sentences: **why can a data set pass a date alignment check and still have a hole in it?** Be specific about what each check actually examines. Finally, say what you would tell a colleague who had already built a chart from this series without running the second check.

4. One paragraph.

Why a monthly mean and a month-end observation answer different questions, and a business situation where choosing the wrong one would lead somewhere costly.

> Notice what is not on this list. You are not submitting code you did not write, or a data file I gave you. Every item asks what you established, chose, or concluded — which is the part of this work that is actually yours, and the only part I can grade.

Average US State Unemployment Rate

The Census Bureau designates US States into defined regions and divisions. Since the 1950s, this has included four statistical regions with 9 subdivisions. This list is available on Wikipedia.

```
state_regions = {
    "new_england":        ["CT", "ME", "MA", "NH", "RI", "VT"],
    "mid_atlantic":       ["NJ", "NY", "PA"],
    "east_north_central": ["IL", "IN", "MI", "OH", "WI"],
    "west_north_central": ["IA", "KS", "MN", "MO", "NE", "ND", "SD"],
    "south_atlantic":     ["DE", "FL", "GA", "MD", "NC", "SC", "VA", "WV"],
    "east_south_central": ["AL", "KY", "MS", "TN"],
    "west_south_central": ["AR", "LA", "OK", "TX"],
    "mountain":           ["AZ", "CO", "ID", "MT", "NV", "NM", "UT", "WY"],
    "pacific":            ["AK", "CA", "HI", "OR", "WA"],
}

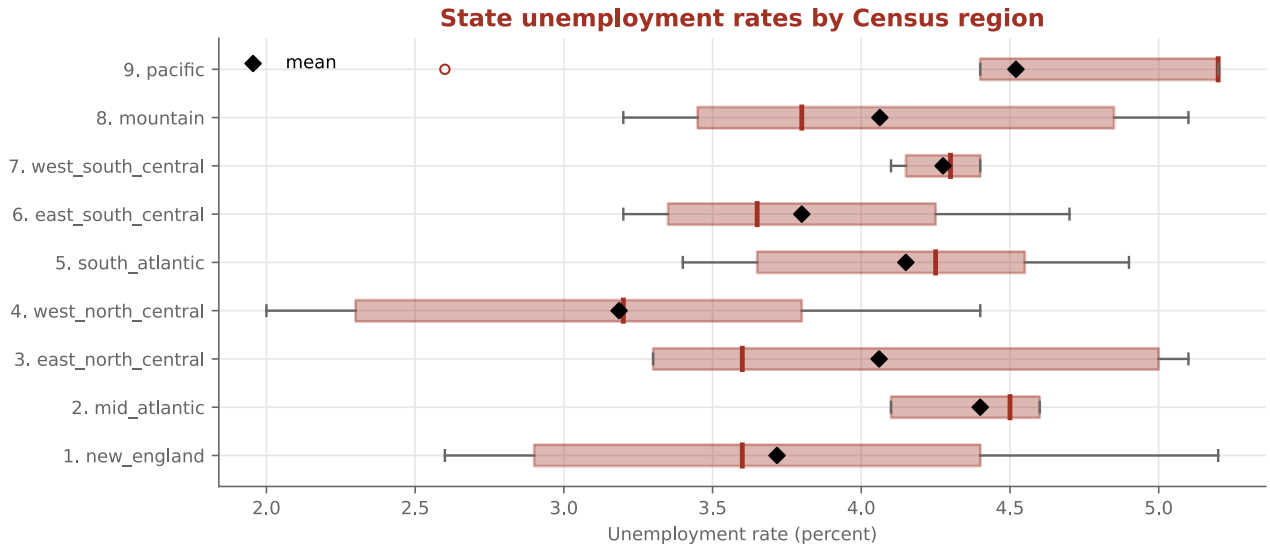
lookup = {
    st: (f"{i}. {name}", i)
    for i, (name, members) in enumerate(state_regions.items(), start=1)
    for st in members
}

my_datfr["Region"] = my_datfr["State"].map(lambda s: lookup[s][0])
my_datfr["Region_Index"] = my_datfr["State"].map(lambda s: lookup[s][1])

unassigned = my_datfr.loc[my_datfr["Region"].isna(), "State"].tolist()
print(f"States assigned to a region: {my_datfr['Region'].notna().sum()} "
      f"of {len(my_datfr)}")
if unassigned:
    print("Unassigned:", unassigned)
```

```
States assigned to a region: 50 of 50
```

```
regions = (my_datfr.sort_values("Region_Index")["Region"].drop_duplicates()
                   .tolist())
_ = box_plot([my_datfr.loc[my_datfr["Region"] == r, "UR"] for r in regions],
             labels=regions,
             title="State unemployment rates by Census region",
             xlab="Unemployment rate (percent)")
```



Branches in statistics

> The purpose of **descriptive statistics** is to summarize, or display, data to quickly get an overview of the information.

```
sel = selected["federal_funds_rate"].dropna()

fig_m, ax = plt.subplots(figsize=(7, 3.2))
ax.boxplot(sel.values, vert=False, widths=0.5,
           patch_artist=True,
           boxprops={"facecolor": UNLV_SCARLET, "alpha": 0.7},
           medianprops={"color": "white"})
ax.set_xlabel("Federal funds rate (percent)")
ax.set_yticks([])
ax.set_title("Month-start observations since 2020")
fig_m.tight_layout()
plt.show()

sel.describe().round(4)
```

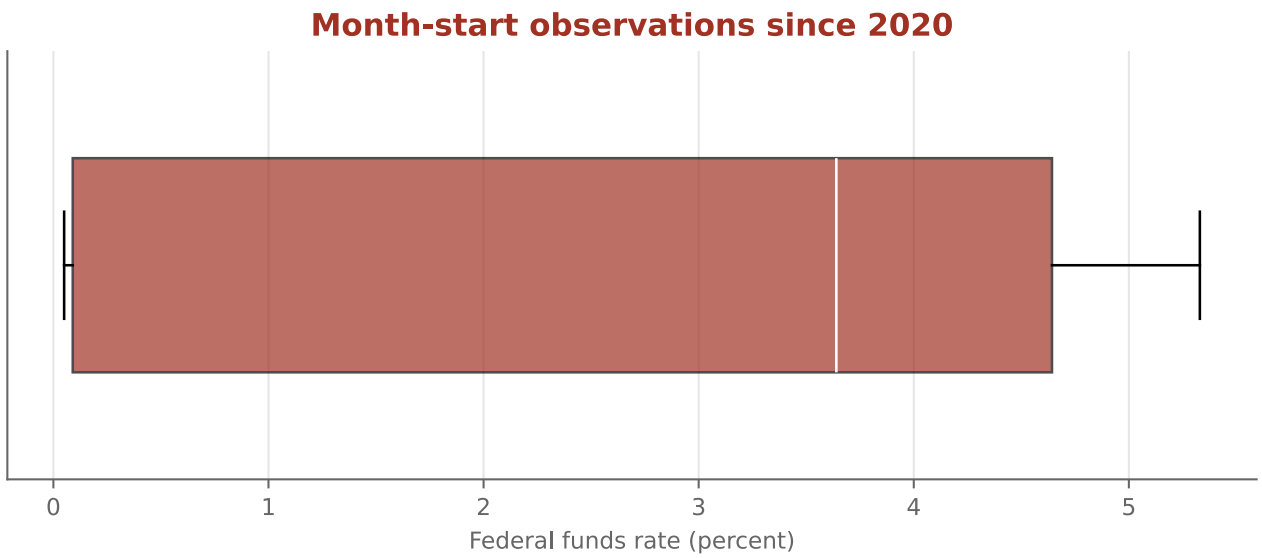


Figure 1: Distribution of the month-start selections.

```
count    80.0000
mean      2.8261
std       2.1342
min       0.0500
25%       0.0900
50%       3.6400
75%       4.6425
max       5.3300
Name: federal_funds_rate, dtype: float64
```

```
full = dff_full["DFF"].dropna()

fig_f, ax = plt.subplots(figsize=(7, 3.2))
ax.boxplot(full.values, vert=False, widths=0.5,
           patch_artist=True,
           boxprops={"facecolor": UNLV_SCARLET, "alpha": 0.7},
           medianprops={"color": "white"})
ax.set_xlabel("Federal funds rate (percent)")
ax.set_yticks([])
ax.set_title("Full daily history")
fig_f.tight_layout()
plt.show()

full.describe().round(4)
```

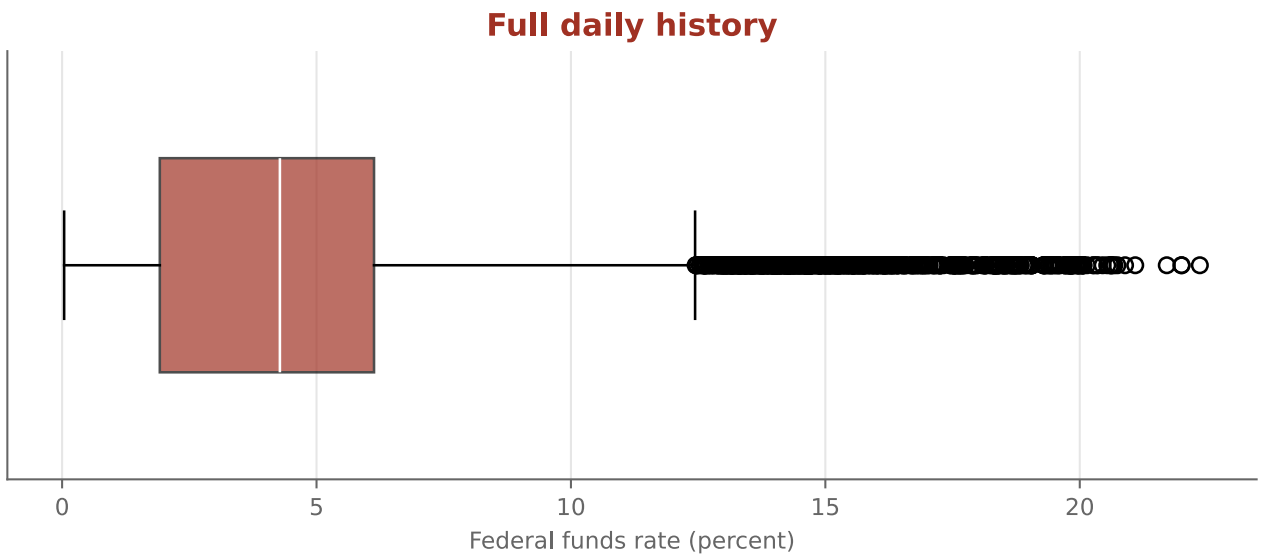
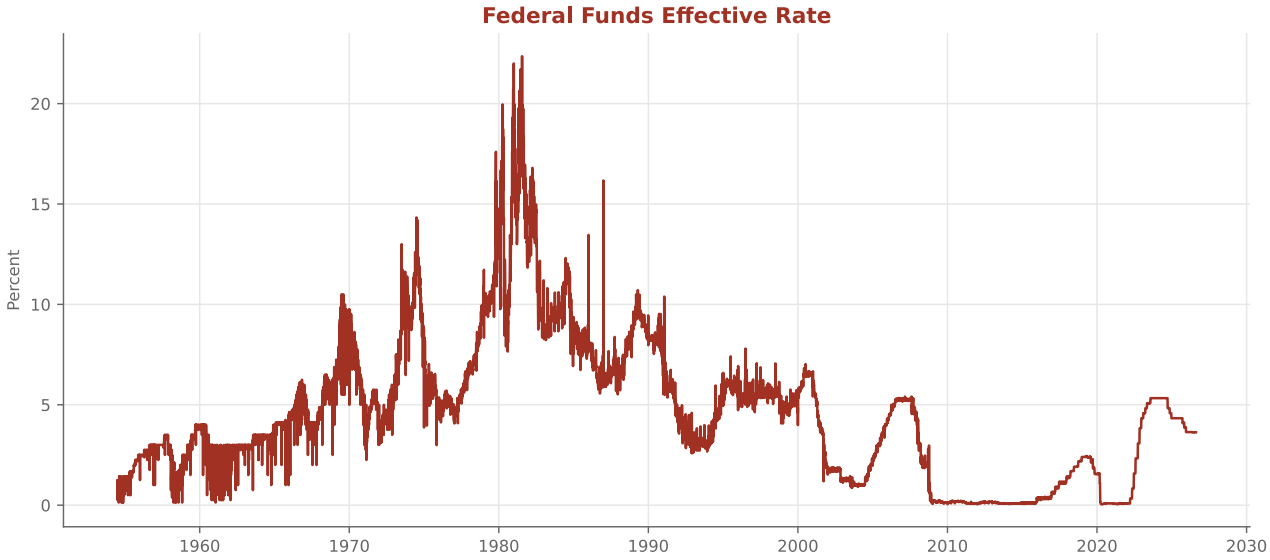


Figure 2: Distribution of the full daily series.

```
count    26342.0000
mean         4.5957
std          3.5453
min          0.0400
25%          1.9200
50%          4.2800
75%          6.1300
max         22.3600
Name: DFF, dtype: float64
```

Descriptive statistics are useful, but have limitations. By summarizing large quantities of data, you lose information. The two summaries above describe the same underlying rate, and they disagree about nearly everything — because they cover different periods. A summary statistic is only as meaningful as the sample behind it.

```
ts = dff_full.reset_index()
_ = time_series(ts["Date"], ts["DFF"],
                title="Federal Funds Effective Rate",
                ylab="Percent")
```



The time series plot shows structure — the 1980s peak, the long descent, the zero lower bound, the recent tightening — that no box plot can express. This is the information the summary discarded.

A **population** represents all possible subjects of interest. A **sample** is a subset of a population.

Inferential statistics allows us to make claims or conclusions about the data and its population.

Inferential statistics at work:

- Production engineers tell us that a properly functioning production line will produce boxes of cereal with a weight of 18.0 oz.
- We select a **random sample** of cereal from boxes coming from this production line (i.e., population).
- We calculate the sample average to be 18.2 oz.
- Using **inferential statistics**, we determine that the probability of observing a sample average of 18.2 oz is less than 1% if the actual population average were 18.0 oz.
- Conclude that the actual population average is greater than 18.0 oz. That is, we find that there is a problem with this production line that needs to be solved.

Predictive statistics analyzes past data to predict future values and make predictions.

Data that describe a characteristic about a population are known as **parameters**.

Data that describe a characteristic about a sample are known as **statistics**.

Business analytics uses statistical analysis and predictive models in the business world to analyze data and make meaningful conclusions about decisions.

Data Analytics uses statistical analysis and predictive models to answer questions in the business world and more broadly to make meaningful conclusions about decisions.

Ethics and statistics — it's a dangerous world of data out there

> A **biased sample** is a sample that does not represent the intended population and that can lead to distorted findings. Biased sampling can occur either intentionally or unintentionally.

One of the most frequent *abuses* of statistics that I notice regularly is the focus on low probability events and the use of these outcomes in the **headlines**. For example, a properly trained data analyst will report a 95% confidence interval for a particular parameter. The analyst may say that there is a 95% chance that the Nevada State net budget will range from $(-\$750 \text{ million}, \$2100 \text{ million})$. It would be *misleading* in an article to say that:

> Researchers find the State of Nevada is in budgetary peril: A budget shortfall of nearly $1 Billion in the cards

When we see a wide ranging 95% confidence interval, the appropriate takeaway is that there is a lot of **uncertainty** and a lot is **unknown** about the processes. I believe journalists and politicians behave like they do because it sells papers and can spur voting interest.

> The same failure mode appears in AI-assisted work. An assistant will produce a confident paragraph whether or not the underlying computation supports it. Confidence in the prose is not evidence about the data. **Check the output.**